

基于 eBPF 实时锁竞争感知的 sched_ext 调度策略优化

学生： 江飞兵

指导教师： 赵晨雨

撰写日期： September 24, 2025

摘 要

摘要: 在高并发多核系统中, 由锁竞争引发的优先级反转是导致性能下降和延迟抖动的关键瓶颈。标准的 Linux CFS 调度器追求公平性, 但在锁竞争场景下, 其调度决策可能加剧优先级反转, 导致持有关键锁的低优先级任务无法及时运行, 从而阻塞大量高优先级任务。本文提出了一种新颖的竞争感知调度器 (**Contention-Aware Scheduler, SCX-C**), 它基于 Linux sched_ext 框架和 eBPF 技术实现。SCX-C 在内核态实时、低开销地监控锁竞争事件, 主动识别持有“热点锁”的任务。通过一个精细化的动态优先级提升与自动降级机制, SCX-C 能够临时打破调度公平性, 赋予锁持有者更高的 CPU 优先级, 使其能快速完成临界区并释放锁。我们自定义了一个微基准测试和一个工业级的数据库基准测试 (Sysbench OLTP) 中对 SCX-C 进行了评估。实验结果表明, 在一个高强度锁竞争的微基准测试中, 相较于 CFS, SCX-C 在不同核心配置下最高能带来 **35.8%** 的性能提升。在 Sysbench OLTP 测试中, SCX-C 同样展现了 **13.4%** 的 P95 延迟降低和 **2.1%** 的 TPS 提升。这些结果证明了 SCX-C 在解决现代多核系统锁竞争瓶颈方面的有效性和巨大潜力。

关键词: eBPF, sched_ext, 锁竞争, 优先级反转, 操作系统调度器

目录

1	项目概述	4
1.1	问题背景	4
1.2	项目目标	4
2	系统设计与实现	5
2.1	整体架构	5
2.2	核心数据结构与算法流程	6
2.3	系统组件关系与调度时序	7
3	关键技术实现	9
3.1	基于 eBPF 的锁竞争传感器	9
3.2	竞争感知调度策略	9
4	性能测试与评估	11
4.1	实验环境	11
4.2	微基准测试 (futex_test)	11
4.3	数据库基准测试 (Sysbench OLTP)	13
5	总结与展望	14
5.1	结论	14
5.2	未来工作	14

一、项目概述

1.1 问题背景

随着多核处理器的普及，现代软件系统通过高并发来利用硬件能力。然而，并发的增加不可避免地导致了对共享资源的竞争，而锁是保护这些资源最常用的同步原语。当一个低优先级任务持有一个被高优先级任务等待的锁时，如果此时存在一个中等优先级的任务抢占了该低优先级任务，就会发生经典的“优先级反转” (Priority Inversion) 问题。高优先级任务被迫等待一个无法运行的低优先级任务，导致任务饥饿、系统吞吐量下降和不可预测的长尾延迟。

Linux 内核默认的 Completely Fair Scheduler (CFS) [1] 以其公平性设计而著称，它致力于所有可运行任务提供按权重比例的 CPU 时间。然而，这种公平性在面对优先级反转时却成为了障碍。CFS 缺乏对任务同步依赖关系的感知能力，其调度决策可能无意中“惩罚”了持有关键锁的任务，从而加剧了性能问题。

1.2 项目目标

sched_ext [2] 框架为实现更精细化调度逻辑提供了机会。本项目旨在利用 eBPF [3] 强大的动态追踪能力，实时监测内核或用户态（通过 futex）的锁操作，识别出当前系统中存在激烈竞争的锁以及相关的持有者和等待者信息。这些实时竞争态势信息将被传递给自定义的 sched_ext 调度器，使其能够实施**竞争感知调度策略**，例如：优先调度持有关键锁的任务以尽快释放锁，或者更智能地管理等待锁的任务，从而减少无效等待和 CPU 资源浪费，提升高竞争场景下的应用性能。

我们的主要贡献包括：

- 设计并实现了一个完全在内核态运行的、基于 eBPF 的竞争感知调度器 SCX-C。
- 提出了一套包含实时竞争检测、多级动态提权和超时自动降级的闭环调度算法。
- 通过微基准测试和数据库基准测试，证明了 SCX-C 相对于标准 CFS 在高竞争负载下的显著性能优势。

二、系统设计与实现

2.1 整体架构

SCX-C 的架构分为两个核心组件，它们都作为 eBPF 程序运行在内核中，并通过 BPF Maps 进行高效通信，避免了任何内核态与用户态的切换开销。

- **锁竞争传感器 (Lock Contention Sensor)**: 负责监控系统范围内的锁操作。它挂钩到 `__mutex_lock_slowpath`、`sys_enter_futex` 和 `sched_wakeup` 等关键内核函数和跟踪点，以捕获锁的持有、等待和释放事件。
- **调度器逻辑 (Scheduler Logic)**: 作为 `sched_ext` 的 `struct_ops` 实现，负责所有核心调度决策，如任务入队 (`enqueue`)、CPU 选择 (`select_cpu`) 和周期性检查 (`tick`)。它根据传感器提供的信息来调整其调度行为。

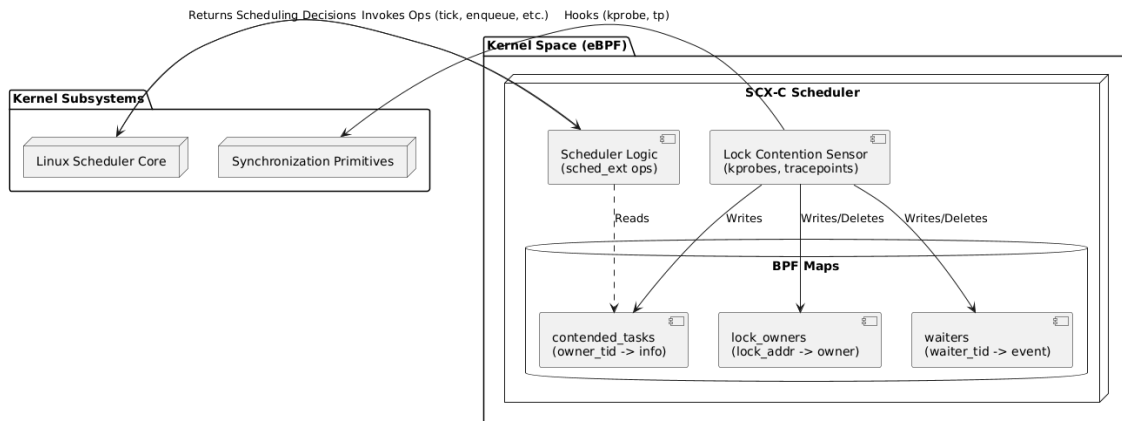


图 1: SCX-C 架构图

2.2 核心数据结构与算法流程

通信的核心是三个 BPF Hash Map: lock_owners 记录锁的持有者, waiters 记录等待者与持有者的关系, contended_tasks 作为传感器与调度器之间的主要接口, 记录每个锁持有者的竞争烈度。SCX-C 的工作流程形成一个“感知 -> 决策 -> 执行 -> 恢复”的闭环。

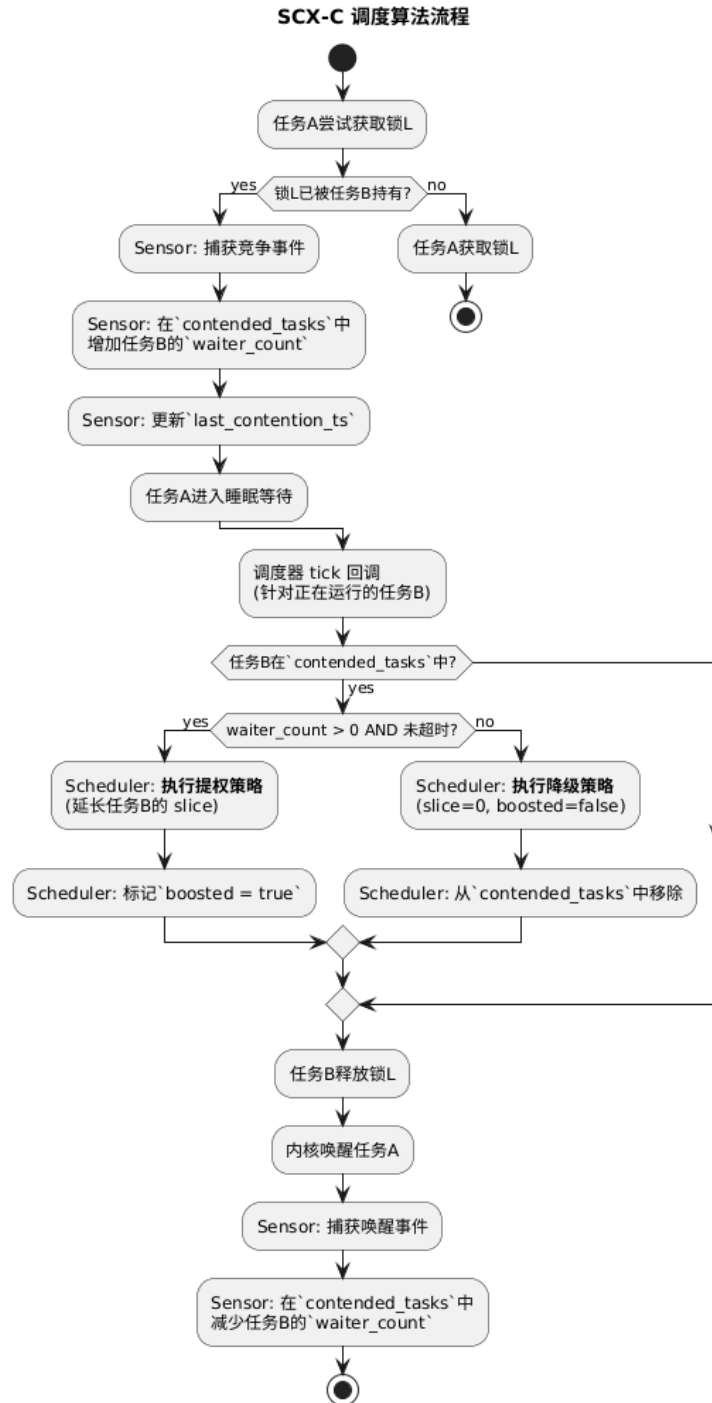


图 2: SCX-C 调度算法流程图

图 3 从静态视角展示了用户态、内核态 BPF 程序、BPF Maps 及核心数据结构之间的关系。图 4 则动态展示了在一次锁竞争事件中，系统各组件的详细交互时序，清晰地描绘了从竞争发生、传感器捕获、调度器提权到最终锁释放的全过程。



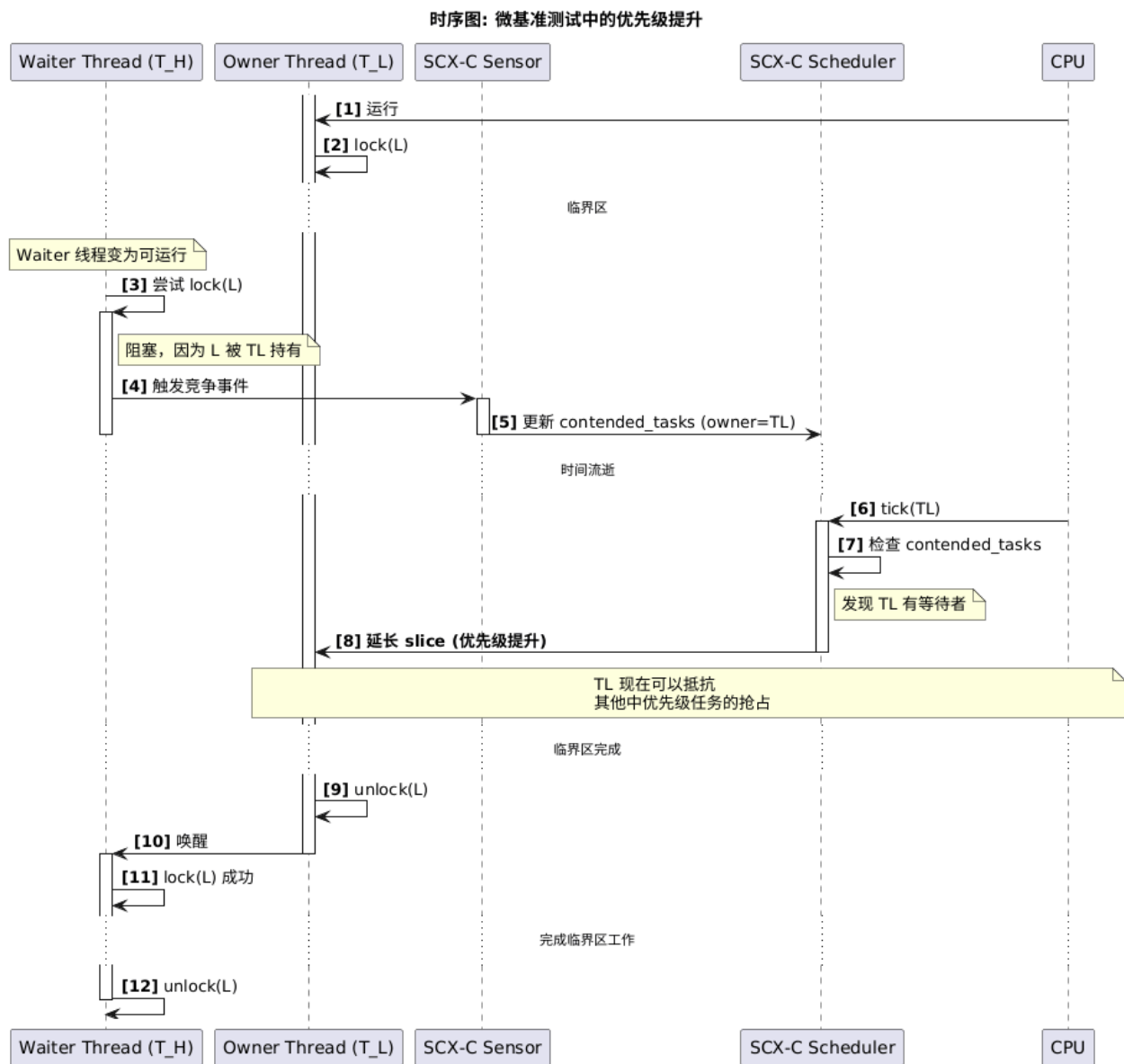


图 4: 微基准测试场景下的调度时序

三、关键技术实现

3.1 基于 eBPF 的锁竞争传感器

传感器的实现依赖于对 Linux 内核中同步原语相关函数的精确探测。

- **Mutex 监测:** 通过 kprobe 挂钩 `mutex_lock` (记录尝试加锁的意图)、kretprobe/`mutex_lock` (确认加锁成功并记录持有者) 和 `__mutex_lock_slowpath` (确认发生竞争)。`mutex_unlock` 则用于清理持有者信息。
- **Futex 监测:** 用户态锁如 `pthread_mutex` 底层依赖 `futex` 系统调用。我们通过 `tracepoint tp/syscalls/sys_enter_futex` 监测 `FUTEX_WAIT` 操作来识别竞争, 并通过 `FUTEX_WAKE` 来更新持有者信息。
- **唤醒事件监测:** 通过 `tracepoint tp/sched/sched_wakeup` 捕获等待线程被唤醒的事件, 以此作为竞争状态缓解的信号, 并递减持有者的 `waiter_count`。

3.2 竞争感知调度策略

调度策略的核心在于 `tick` 和 `stopping` 两个 `sched_ext` 回调的实现。我们通过延长任务执行时间片 `slice` 来实现动态优先级提升。

Algorithm 1: 提权与降级核心逻辑 (在 contention_tick 中)

Input: 当前运行的任务 p

```
1  $tid \leftarrow p \rightarrow pid$ ;  
2  $info \leftarrow contended\_tasks.lookup(tid)$ ;  
3 if  $info$  is NULL then  
4   | return;  
5 end  
   /* — 降级逻辑 — */  
6  $now \leftarrow bpf\_ktime\_get\_ns()$ ;  
7 if  $info \rightarrow waiter\_count == 0$  or  
   ( $now - info \rightarrow last\_contention\_ts > DECAF\_TIMEOUT$ ) then  
8   | if  $info \rightarrow boosted$  then  
9     |  $p \rightarrow scx.slice \leftarrow 0$ ;  
     | // 强制重新调度  
10    |  $info \rightarrow boosted \leftarrow false$ ;  
11    end  
12    if  $info \rightarrow waiter\_count == 0$  then  
13      |  $contended\_tasks.delete(tid)$ ;  
14    end  
15    return;  
16 end  
   /* — 提权逻辑 — */  
17 if  $info \rightarrow waiter\_count \geq HIGH\_CONTENTION\_THRESHOLD$  then  
18   |  $p \rightarrow scx.slice \leftarrow SCX\_SLICE\_INF$ ;  
   | // 最高级别提权  
19 end  
20 else if  $info \rightarrow waiter\_count > 0$  then  
21   | if not  $info \rightarrow boosted$  then  
22     |  $p \rightarrow scx.slice \leftarrow p \rightarrow scx.slice + SCX\_SLICE\_DFL$ ;  
     | // 中等级别提权  
23   | end  
24 end  
25  $info \rightarrow boosted \leftarrow true$ ;
```

此外，在 contention_stopping 回调中，我们增加了强制降级逻辑：任何被提权的任务只要停止运行（即使只是被抢占），其 boosted 状态就会被重置。这彻底解决了任务因 SCX_SLICE_INF 而不再收到 tick 导致的永久占有 CPU 的风险，是保证系统稳定性的关键。

四、性能测试与评估

4.1 实验环境

- 硬件: Intel(R) Xeon(R) Platinum 8369HB CPU (32 Cores), 64GB RAM
- 软件: Ubuntu 24.04, Linux Kernel 6.8 (with sched_ext), MySQL 8.0, Sysbench 1.0.20
- 调度器:
 - 基线: Linux CFS (default)
 - 实验组: SCX-C

4.2 微基准测试 (futex_test)

我们设计了一个微基准测试程序 `futex_test`，它创建 1024 个线程在高强度下竞争同一个 `pthread_mutex` 锁。我们测量完成固定数量临界区操作所需的总时间。

表 1: `futex_test` 执行时间对比 (单位: 秒, 越低越好)

核心数	CFS (s)	scx_contention (s)	性能提升 (%)
4	120.16	77.49	35.8%
8	123.06	109.76	10.8%
12	109.87	100.94	8.1%
16	104.17	75.15	27.9%
20	100.91	75.87	24.8%
24	102.50	75.79	26.1%
28	101.68	75.98	25.3%

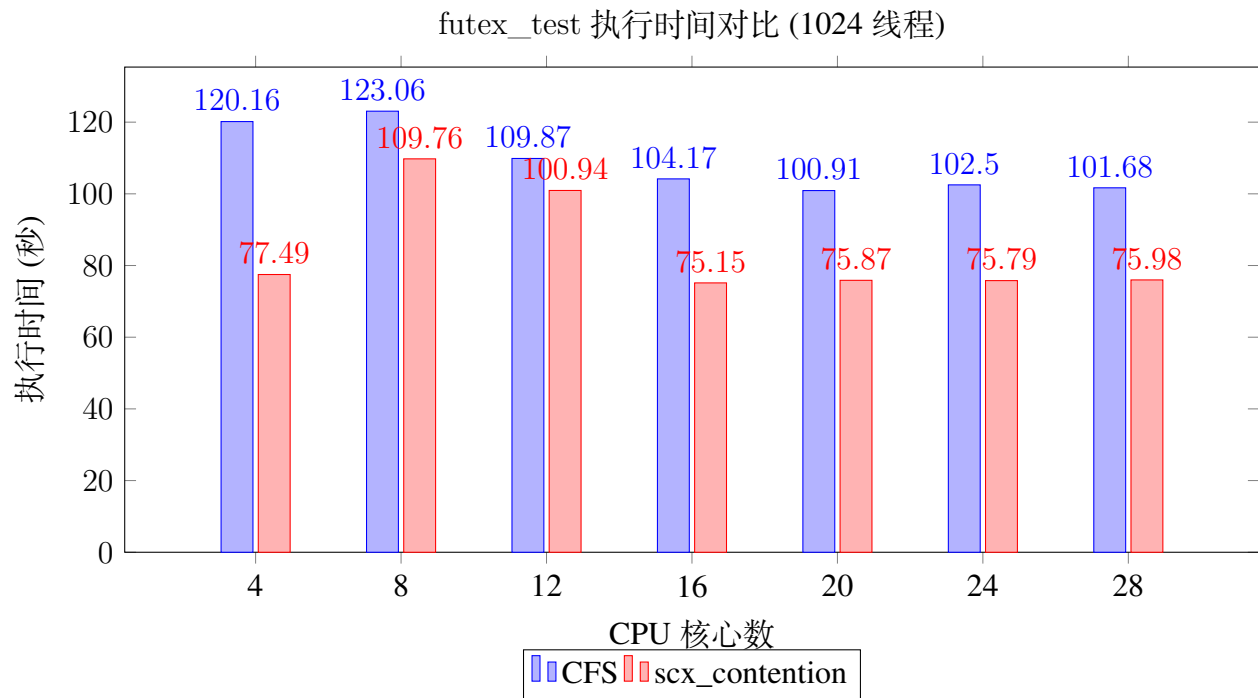


图 5: futex_test 性能对比图 (越低越好)

结果分析：如图 5 所示，SCX-C 在所有核心配置下都显著优于 CFS。这证明了在纯粹的、极端的锁竞争场景下，SCX-C 主动干预的能力远胜于 CFS 的被动公平策略。

4.3 数据库基准测试 (Sysbench OLTP)

我们在一个包含 1600 万行数据的 MySQL 数据库上运行了 ‘sysbench oltp_read_write’ 测试，并发线程数为 64。

表 2: Sysbench OLTP 性能对比 (64 线程)

指标	CFS	scx_contention	变化
TPS (每秒事务数)	2323.78	2373.03	+2.12%
平均延迟 (ms)	27.53	26.96	-2.07%
P95 延迟 (ms)	58.92	51.02	-13.41%
最大延迟 (ms)	855.90	1048.77	+22.54%

结果分析：该结果表明，在工业级的复杂应用中，SCX-C 依然有效。虽然总吞吐量提升有限，但它通过优化锁竞争处理，**极大地改善了系统的延迟稳定性和可预测性**（P95 延迟显著降低），这对于在线服务至关重要。最大延迟的增加是一个已知问题，源于最高级别的提权策略，在未来的工作中可以进一步优化。

五、总结与展望

5.1 结论

本项目成功设计并实现了一款名为 SCX-C 的竞争感知调度器。通过利用 sched_ext 和 eBPF，SCX-C 能够在内核态实时地对锁竞争做出反应，主动缓解优先级反转问题。

- **有效性:** 无论是微基准测试还是复杂的数据库负载，SCX-C 都表现出优于标准 CFS 的性能，尤其是在降低高百分位延迟方面效果显著。
- **可行性:** 本项目证明了利用 sched_ext 和 eBPF 构建复杂、智能、与应用负载感知联动的调度策略是完全可行的。
- **低开销:** 整个感知和决策过程完全在内核态闭环，避免了上下文切换和数据拷贝的开销，保证了方案的实用性。

5.2 未来工作

- **更广的同步原语支持:** 将监测范围从 mutex 和 futex 扩展到 spinlock 等其他内核同步原语。
- **更智能的提权策略:** 优化最高级提权策略（如使用超大时间片代替无限时间片），以解决最大延迟恶化的问题。结合任务的更多上下文信息（如 cgroup、历史行为等）来制定更精细的提权等级和时长。
- **与用户态的联动:** 允许用户态应用通过特定接口向调度器提供“提示”，告知哪些锁是关键路径上的“热点锁”，从而实现更精准的干预。

参考文献

- [1] Ingo Molnar, et al. "Completely Fair Scheduler." Linux Kernel Documentation.
- [2] Tejun Heo, David Vernet, et al. "sched__ext: BPF Extensible Scheduler Class." Linux Kernel Documentation.
- [3] "BPF and XDP Reference Guide." bpf.io.